

PRD v5.0 — BharatOpt Core: A From-Scratch Sovereign LP / QP / MILP Solver with GPU Acceleration

SIH26119 · MRPL · Re-baselined to the problem statement · 30 Sep 2026

***Status.** Design document. Nothing here is implemented or benchmarked yet. Performance figures are **[TARGET]** (goals to be measured), not claims. **[VALIDATE]** = confirm with MRPL/SIH mentors. **[VERIFY]** = not confirmed from a primary source. **[v1.x]/[v2.0]** = deferred.*

0. What changed from v4.0 and why

v4.0 was a strong *refinery planning application* built **on top of** SCIP, SoPlex, PaPILO, HiGHS, cuPDLPx and cuOpt. Problem Statement 26119 asks for something different: a **solver core, built from scratch from mathematical foundations, not built upon any existing open-source solver library**. v5.0 therefore inverts the priorities.

Topic	v4.0	v5.0
Center of gravity	Refinery planning product + XAI + 3D UI	Solver engine (LP, QP,

Topic	v4.0	v5.0, API/CLI
Solver kernels	Integrate SCIP/HiGHS/SoPlex/PaPILO/cuPDLPx	Write our own ; third-party solvers only as <i>external reference oracles</i> in the test harness, never linked or shipped
QP	Non-goal	In scope (convex QP in v1.0)
Extensibility	Not a goal	Architecture must admit MIQP / NLP / MINLP (plugin points, expression layer)
Benchmarks	MRPL-class instances	Netlib, MIPLIB 2017, Mittelmann, QPLIB + refinery case studies from open literature
Robustness	Verifier gate	Verifier gate plus explicit

2. **Competitive with established open-source solvers on defined benchmark subsets** [TARGET, §11],
3. **Transparent and extensible** (something Indian developers can inspect, modify and extend),
4. **Faster where structure or GPUs help** (batched what-ifs, first-order LP at scale, GPU heuristics), measured, not assumed.

Trust architecture (four layers, never conflated; retained):

Layer	Artifact	Answers
1 Verifier gate	Independent feasibility / optimality / bound checks	"Is this answer valid?"
2 Sensitivity & explanation	Duals, reduced costs, ranging, IIS, plan diff	"Why this? Why not X?"
3 Search trace	Optional B&B tree log / view	"What did the solver do?" (process, not cause)
4 Comparison mode	Side-by-side vs reference solver / incumbent tool	"How does it compare to what we trust?"

2. Compliance matrix against SIH26119

PS requirement	Where addressed	Acceptance
Sovereign solver core , not a modeling environment	§1, §5, §7	Core library + CLI/API only; thin model layer
LP, MILP, QP as initial focus	FR-LP, FR-QP, FR-MIP	Netlib / MIPLIB / QPLIB gates (§11)
Extensible to MIQP, NLP, MINLP	§7.4, §5 FR-EXT	Plugin interfaces + convex MIQP prototype [v1.5]; NLP/MINLP [v2.0]

PS requirement	Where addressed	Acceptance
Revised simplex + interior point	FR-LP1..8	Own implementations
B&B, B&C, cuts, presolve, heuristics, node selection	FR-MIP1..12	Own implementations
Sparse techniques, numerical linear algebra	§7.2, FR-LA	Own sparse LU / Cholesky / ordering
Multi-core parallelism	FR-PAR	Deterministic parallel mode (T1)
GPU where measurable benefit	FR-GPU	Enabled only where benchmark shows benefit
Not built on existing open-source solver library; from scratch	§3 Clean-room policy	Dependency audit + SBOM gate (G0)
Degenerate, ill-conditioned, weak-relaxation robustness	FR-ROB, §11 robustness suite	Published demonstration report
Thousands to millions of variables/constraints	NFR-SCALE	Scale ladder test (§11)
Benchmarks: MIPLIB, Netlib, Mittelmann	§11	Comparison vs ≥ 1 established solver (we use ≥ 2)
Refinery scheduling, blending, production planning, logistics, power dispatch, supply chain	§8 application benchmark suite	Open-literature models
Basic API or CLI sufficient; GUI not required	FR-API	CLI, C API, Python binding

3. Clean-room / "from scratch" policy (new, critical)

Rule. No source code, algorithm implementation, or compiled library from any existing optimization solver (open or commercial) is linked into, copied into, or shipped with BharatOpt Core.

Category	Policy
Solver libraries (SCIP, SoPlex, PaPILO, HiGHS, CBC, GLPK, cuOpt, cuPDLP*, OSQP, Ipopt, etc.)	Forbidden in the product build. Allowed only as external reference oracles in the test harness, run as separate processes for result comparison. Not distributed.
Algorithms	Implemented from published mathematics (textbooks, papers). Each module has a design note citing sources. No reading-and-transcribing of solver source; keep a provenance log.
Standard language runtime & OS libs (C++ STL, threads, CUDA runtime/driver)	Allowed.
Dense BLAS/LAPACK, vendor GPU math libraries (cuBLAS, cuSPARSE, cuSOLVER)	[CLARIFY with MRPL/SIH] These are numerical primitives, not optimization solvers. Recommended stance: own sparse kernels on the critical path; vendor primitives allowed as <i>swappable</i> backends behind an internal interface, off the determinism-critical path. If the jury reads "from scratch" strictly, ship the own-kernel build.
File-format parsers (MPS, LP, QPS)	Own parser.
Licensing of our code	Choose permissive licence [VALIDATE with MRPL legal].

Gate G0 (Phase 0): automated dependency scan + SBOM shows zero solver-library dependencies in the product target.

4. Users and needs

Persona	Need	Acceptance signal
Optimization engineer / OR developer (primary)	Robust LP/QP/MILP engine with inspectable internals, C/Python API	Solves benchmark sets; readable logs; extension example
Refinery planner / scheduler (MRPL)	Crude selection, blending, tank/CDU schedule models solved reliably	Replays ≥ 3 historical months within agreed tolerance [VALIDATE]
Process engineer	Realistic unit limits, sensitivity	Sensitivity report matches finite differences
HSE / emissions	Emission caps in plan	Carbon-cap sweep
IT/OT	Secure, offline, supportable	Air-gapped install, SBOM, audit log
Academic / strategic user	Transparent base for future Indian solvers	Modular code, docs, extension guide
Management	Licence-cost avoidance and sovereignty	Comparison report with verifier status

5. Functional requirements (M = must, S = should, C = could)

5.1 Problem classes and I/O

ID	Requirement	Pri
----	-------------	-----

ID	Requirement	Pri
FR-IO1	Read/Write MPS (fixed & free), LP format, QPS; MPS/LP round-trip tests	M
FR-IO2	Internal model: sparse column/row storage, bounds, integrality, quadratic objective term $\frac{1}{2}x^T Q x$, SOS1/2 [S], indicator constraints [C]	M
FR-IO3	Deterministic model hash (same input $\rightarrow$ same hash)	M
FR-IO4	Input validation with actionable messages (units, ranges, consistency)	M

5.2 Linear algebra (own implementation)

ID	Requirement	Pri
FR-LA1	Sparse LU with Markowitz pivoting + threshold partial pivoting; hyper-sparse triangular solves	M
FR-LA2	Basis update: Forrest–Tomlin (or product-form fallback) with stability-triggered refactorization	M
FR-LA3	Sparse Cholesky / LDL^T with fill-reducing ordering (AMD-style minimum degree; nested dissection [S])	M
FR-LA4	Scaling (geometric + equilibration / Ruiz), coefficient-range diagnostics	M
FR-LA5	Iterative refinement, condition-estimate monitoring	S

5.3 Presolve

ID	Requirement	Pri
FR-PRE1	Singleton rows/cols, empty/duplicate rows and cols, fixed variables, bound tightening	M
FR-PRE2	Dominated columns, doubleton equations/aggregation, implied-free columns	M
FR-	MIP presolve: coefficient tightening, probing, clique detection, implied	S

PRE3	Requirement: parallel/dominated rows	Pri
FR- PRE4	Full postsolve (primal, dual, basis) with verifier check	M

5.4 Linear programming

ID	Requirement	Pri
FR- LP1	Dual simplex (revised, bounded), dual steepest-edge pricing, bound-flipping ratio test	M
FR- LP2	Primal simplex with steepest-edge/devex pricing; Phase 1 via composite/artificial approach	M
FR- LP3	Degeneracy handling : cost/bound perturbation with clean-up, Harris ratio test, anti-cycling guarantee	M
FR- LP4	Interior-point method : primal-dual predictor–corrector (Mehrotra), homogeneous self-dual embedding for infeasibility/unboundedness detection	M
FR- LP5	Crossover from IPM to a basic solution	M
FR- LP6	Warm start from basis; ray (unbounded) and Farkas certificate (infeasible) extraction	M
FR- LP7	Sensitivity: duals, reduced costs, cost/bound/RHS ranging from basis	M
FR- LP8	IIS finder for infeasible LP (deletion / elastic filter)	S

5.5 Quadratic programming (new vs v4.0)

ID	Requirement	Pri
FR- QP1	Convex QP via primal-dual IPM reusing FR-LA3 (KKT system, quasi-definite regularization)	M
FR- QP2	Convexity check (Cholesky/eigen-based) with clear status if nonconvex	M

ID	Requirement	Pri
FR- QP3	QP presolve and warm start; active-set or simplex-type QP method [v1.5]	S
FR- QP4	Convex MIQP via B&B with QP relaxations [v1.5]; nonconvex QP via spatial B&B [v2.0]	S
FR- QP5	Second-order cone extension point [v2.0]	C

5.6 Mixed-integer programming

ID	Requirement	Pri
FR- MIP1	LP-based branch-and-bound with best-bound + depth-first hybrid node selection, plunging	M
FR- MIP2	Branching: most-fractional (baseline), pseudocost, reliability branching with strong-branching budget	M
FR- MIP3	Cutting planes at root and tree: Gomory mixed-integer, MIR / c-MIR, knapsack cover, lifted cover, flow cover, clique, implied-bound, zero-half	M (Gomory, MIR, cover, clique); S (rest)
FR- MIP4	Cut management: efficacy, parallelism and density filters, aging, pool	M
FR- MIP5	Primal heuristics: rounding, diving (fractional, guided, coefficient), feasibility pump, feasibility jump , RINS/RENS, local branching	M (rounding, diving, FP); S (rest)
FR- MIP6	Conflict analysis, reduced-cost fixing, node presolve/propagation	S
FR- MIP7	Restarts; symmetry detection & orbital fixing / symmetry-breaking rows	S
FR- MIP8	Exactly-tracked incumbent, best valid bound, gap, node count; status codes (§9.3)	M
FR- MIP9	Solution pool (top-K)	S
FR-	Limits by work units / nodes / iterations (always); wall-clock	M

MIP10 ID	only in T0 Requirement	Pri
FR- MIP11	Callback/plugin API for user cuts, heuristics, branching rules	S
FR- MIP12	Anytime KPI logging: incumbent-vs-time, bound-vs-time, time-to-1%-of-best-known	M

5.7 Parallelism and GPU

ID	Requirement	Pri
FR- PAR1	Multi-core: parallel node processing with deterministic mode (fixed work partition, ordered synchronization)	M
FR- PAR2	Concurrent portfolio (different strategies/seeds) in T0; scenario fan-out for batch what-ifs	S
FR- GPU1	GPU first-order LP (PDHG / PDLP-style with restarts, adaptive step size, scaling), written from published math, CUDA	S
FR- GPU2	GPU heuristics for MIP: feasibility jump, fix-and-propagate, parallel diving	S
FR- GPU3	Batched small-LP solves for strong branching / scenarios on GPU	C
FR- GPU4	Benefit rule: GPU path is enabled by the router only for instance classes where benchmarks show gains; else CPU. Never assumed faster.	M
FR- GPU5	GPU LP results never reported as bounds directly; a valid dual bound is recomputed on CPU from the multipliers (§7.3)	M
FR- GPU6	Automatic GPU→CPU fallback	M

5.8 Robustness

ID	Requirement	Pri
FR- ROB1	Explicit tolerances (§9.1); no hidden defaults	M

FR- ROB2	Requirement: Multiple ladder: rescale → perturb → switch algorithm (primal/dual/IPM) → increase refactor frequency → report NUMERICAL	Pri
FR- ROB3	Process isolation option and watchdog; crash never loses last verified incumbent	M
FR- ROB4	Degeneracy, ill-conditioning and weak-relaxation demonstration suite with published results	M
FR- ROB5	Fuzzing with random model generator and delta-debugging to minimize failing instances	M

5.9 Verification, reproducibility, explanation (retained)

ID	Requirement	Pri
FR- VER1	Independent verifier (§9.2): primal feasibility, integrality, objective recomputation, bound check, KKT for LP/QP	M
FR- VER2	Forward-simulation verifier for scheduling models (tank balances, specs)	M
FR- DET1	Run tiers T0 opportunistic / T1 deterministic (default for published results) / T2 certified replay (§9.3)	M
FR- DET2	Run archive: inputs, model hash, versions, parameters, seeds, logs, replay bundle	M
FR- CERT1	Optional certified-optimality mode: emit a checkable B&B proof log with an independent checker (own format)	C
FR-X1	Duals, reduced costs, ranging on fixed-integer LP, tagged <code>local_at_incumbent</code>	M
FR-X2	Breakeven price for non-slate variables (LP exact; spot/binary by re-solve)	S
FR-X3	"Why not X?" via objective-cutoff + IIS (X-MILP style) at LP-relaxation level	S
FR-X4	Forced-scenario plan diff with objective delta	S
FR-X5	Template narration (no LLM) of structured explanations	C

5.10 Interfaces

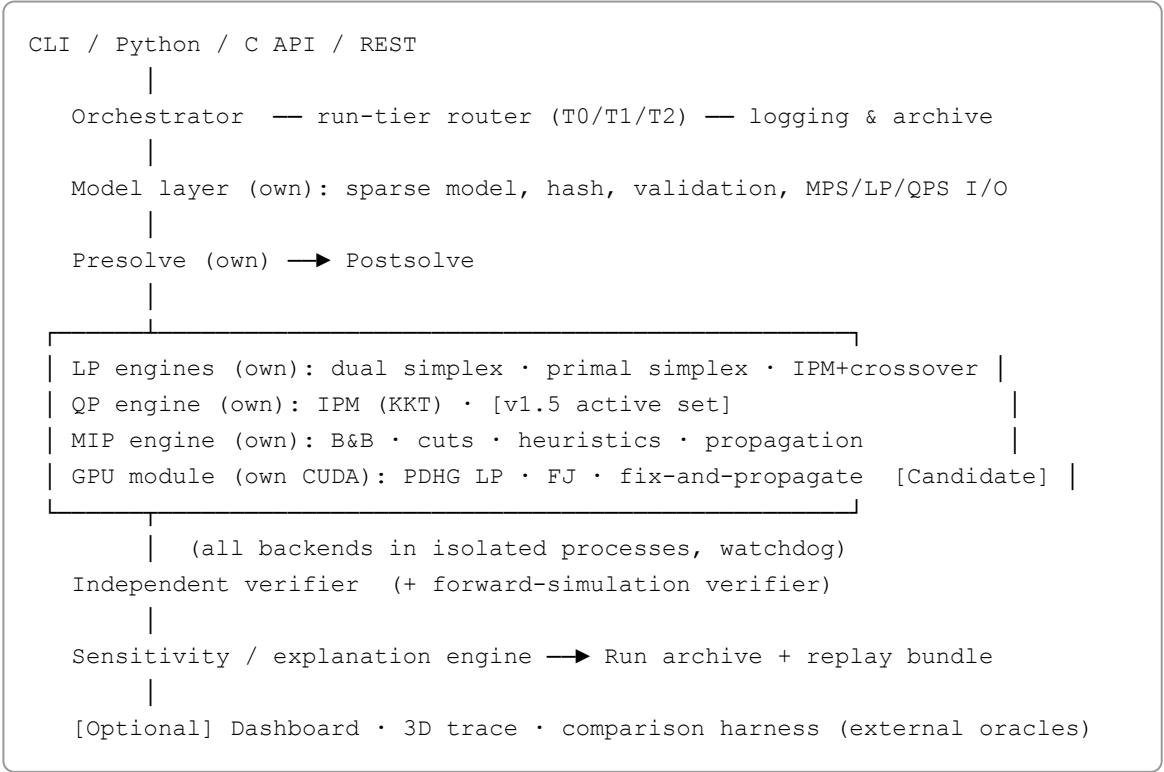
ID	Requirement	Pri
FR-API1	C API (stable ABI) and C++ core	M
FR-API2	Python binding (build, solve, query, callbacks)	M
FR-API3	CLI: <code>bharatopt model.mps --tier T1 --log run.json</code>	M
FR-API4	Thin algebraic modeling helper in Python (variables, constraints, expressions)	S
FR-API5	REST wrapper for service deployment	C
FR-UI1	Optional web dashboard: gap/bound chart, log viewer, verifier checklist, plan diff (2D)	C
FR-UI2	Optional 3D search-trace view, labelled "process, not cause"	C [v1.5]
FR-CMP1	Comparison mode: run same model on BharatOpt and external reference oracles, report objective/time/gap side by side	M

5.11 Extensibility (PS requirement: future MIQP/NLP/MINLP)

ID	Requirement	Pri
FR-EXT1	Plugin interfaces: relaxation provider, cut separator, branching rule, primal heuristic, propagator, presolver	M
FR-EXT2	Expression-graph layer (DAG of nonlinear expressions with interval bounds and automatic differentiation) as internal module	S [v1.5 design, v2.0 use]
FR-FXT3	Spatial B&B with McCormick/ α BB-style convex relaxations for bilinear terms	S [v2.0]; prototype on

EXT3	Simultaneous	prototype on
ID	Requirement	Task blending
FR-	NLP local solver (e.g., interior-point with exact Hessians)	C
EXT4	[v2.0]	

6. Architecture



Principles (retained and extended):

1. Published results come only from CPU **Production-maturity** engines. GPU and experimental modules may *propose* (warm starts, incumbents); the verifier disposes.
2. Vertex-dependent outputs (duals, ranging) come from the CPU simplex on the fixed-integer LP (first-order LP methods yield no basis).
3. Each engine runs in its own process with a watchdog; crash → fallback, last verified incumbent preserved.
4. Visualization consumes events and never writes engine state.
5. Every math specification item has a hand-computed test (App. A).

Module maturity tiers (internal, replacing v4.0's third-party tiers):

Tier	Modules	Rule
Production	LP simplex, presolve, sparse LA, B&B, core cuts, verifier	May produce published plans
Candidate	IPM+crossover (until stress-tested), GPU PDHG, GPU heuristics, advanced cuts	Propose only; always verified
Experimental	Nonlinear/spatial B&B, MIQP, certified proof mode	Feature flag; T0 only

Promotion: N regression solves (Netlib + MIPLIB subset + refinery suite; N set with QA) with 0 verifier failures, 0 crashes, objective agreement within 1e-6 against reference oracles, and fuzz campaign clean.

7. Mathematical and algorithmic specification

7.1 Canonical form (single, used everywhere)

```

max  cTx - ½xTQx      (Q = 0 for LP/MILP; Q ≥ 0 for convex QP in max form with c)
s.t.  A x ≤ b          (duals y ≥ 0)
      E x = d          (duals λ free)
      l ≤ x ≤ u        (some xj integer)

```

Engine converts min↔max internally by negating the objective.

7.2 Linear-algebra kernel notes

Basis factorization $B = LU$ with Markowitz count and threshold $\tau \in [0.01, 0.1]$; refactor every K updates or when growth/residual test fails. Cholesky of $A \triangleright A^T$ (or augmented KKT) with regularization δ when pivots are tiny. Ordering chosen once per IPM run (symbolic factorization reuse across iterations).

7.3 Valid dual bound (max form) — derived, retained

For any feasible x , any $y \geq 0$, any λ : $c^T x \leq c^T x + y^T (b - Ax) + \lambda^T (d - Ex) = b^T y + d^T \lambda + r^T x$, with $r = c - A^T y - E^T \lambda$. Maximizing $r^T x$ over the box gives $g(y, \lambda) = b^T y + d^T \lambda + \sum_j (r_j^+ u_j - r_j^- \ell_j)$, $r_j^+ = \max(r_j, 0)$, $r_j^- = \max(-r_j, 0)$. If some $r_j > 0$ with $u_j = +\infty$ (or $r_j < 0$ with $\ell_j = -\infty$), $g = +\infty$ (valid, useless). For minimization use the mirrored expression to obtain a lower bound. **A first-order iterate is never a bound; only $g(y, \lambda)$ computed from its multipliers is.** Gap (max): $(UB - LB) / \max(1, |UB|)$, LB = verified incumbent, UB = best valid bound over open nodes.

For convex QP, the Wolfe dual bound $b^T y + d^T \lambda - \frac{1}{2} x^T Q x + (\text{box terms})$ evaluated at the current x is used; PSD of Q is checked.

7.4 MILP algorithm specification (own implementation)

1. **Presolve** (FR-PRE) $\rightarrow$ root LP (dual simplex).
2. **Root cut loop**: separate Gomory/MIR/cover/clique/flow-cover; accept by efficacy and parallelism; stop on stall (gap-closed $< \epsilon$ over R rounds). Cuts computed with safe-rounding (numerically safe MIR) and validity checks on coefficients.
3. **Heuristics at root**: rounding, FP/FJ, diving; incumbent $\rightarrow$ verifier.
4. **Tree search**: node LP warm-started from parent basis; reliability branching; best-estimate with plunging; propagation and reduced-cost fixing per node.
5. **Termination**: gap $\leq$ tolerance, work/node limit (T1), or infeasible/unbounded proof.
6. **Bound safety**: node bounds accepted only when the LP solve is verified (dual feasibility residual $\leq \text{tol}$) — otherwise the node is re-solved with tighter settings or its parent bound is kept.

7.5 Extensibility path to nonconvex problems [v2.0]

Bilinear $w = x \cdot y$ relaxed by McCormick envelopes: $w \geq x^L y + y^L x - x^L y^L$, $w \geq x^U y + y^U x - x^U y^U$, $w \leq x^U y + y^L x - x^U y^L$, $w \leq x^L y + y^U x - x^L y^U$; spatial branching on the variable with largest relaxation gap; bound tightening (OBBT) as optional. Used first on mixed-grade tank blending.

8. Application benchmark suite (refinery and industrial)

The PS lists refinery scheduling, crude blending, process optimization, production planning, logistics, power dispatch, transport, supply chain. We ship an **open-literature model library** on the core (Python helper), used for both benchmarking and demonstration.

Model	Type	Source of data
L1 Monthly crude selection & blending (§8.1)	MILP (few binaries, LP-dominated)	Synthetic + public assay-style data
L2 Tank/CDU scheduling (§8.2, single-grade)	MILP (hard scheduling)	Open literature scheduling benchmarks
L2-B Mixed-grade tanks (bilinear)	nonconvex MINLP	Open literature; [v2.0] with own spatial B&B
Unit commitment / economic dispatch	MILP / convex QP	Open power-system test cases
Multi-echelon supply chain, facility location	MILP	Open literature
Portfolio-type QP, blending with quadratic cost	QP	QPLIB / literature
Transportation, assignment, network flow	LP / MILP	Netlib / MIPLIB

Real MRPL data is **[VALIDATE]**; all public results use synthetic/open data. Facts about MRPL (nameplate 15 MMT/PA $\approx$ 300 kbpd; FY2022-23 throughput 17.14 MMT, i.e. above nameplate; ONGC subsidiary at $\sim$ 71.6%; Nelson index reported $\approx$ 10.6–11.3; 250+ crude types) come from v4.0's verification log and are used only to size synthetic instances. **Capacity bounds are data, never hard-coded.**

8.1 Level 1 — monthly crude selection (retained)

Variables: $x_{jc} \geq 0$ (kbbbl crude $j \rightarrow$ CDU c), $s_j \geq 0$ closing stock, $y_i \in \{0, 1\}$ spot-cargo purchase, $w_p \geq 0$ product output. Objective: $\max \sum_p \pi_p w_p - \sum_{\{j,c\}} (\kappa_j + \gamma_c) x_{jc} - \sum_{\{j \in J_S\}} f_j y_j - \sum_j h_j s_j$. Constraints:

1. Stock balance: $\sum_c x_{jc} + s_j = s_j^0 + q_j + A_j y_j$.
2. Stock capacity: $\sum_j s_j \leq S_{\max}$.
3. CDU capacity: $\kappa_{\min} \leq \sum_j x_{jc} \leq \kappa_{\max}(\text{data})$

3. CDU capacity: $R_{c \min} \leq \sum_j x_{jc} \leq R_{c \max}(\text{data})$.
4. Allowability: $x_{jc} = 0$ for prohibited crude–CDU pairs (e.g., TAN metallurgy).
5. Yields: $w_p = \sum \eta_{pj} x_{jc}$, with $\sum_p \eta_{pj} \leq 1 + g_j$ on a declared mass or volume basis [VALIDATE yield source].
6. Demand: $D_p^{\min} \leq w_p \leq D_p^{\max}$.
7. Quality per CDU (mass-weighted, linear because crude properties are data): Sulfur $\sum_j (s_j - S_c^{\max}) \rho_j x_{jc} \leq 0$; TAN likewise; **gravity via SG, not API** (API does not blend linearly): $\sum_j (SG_j - SG_c^{\max}) x_{jc} \leq 0$, $SG_c^{\max} = 141.5 / (A_c^{\min} + 131.5)$.
8. Emissions proxy: $\sum e_{fj} x_{jc} \leq E_{\text{cap}}$ (feed-based proxy; limitation §13).

8.2 Level 2 — tank/CDU scheduling, single-grade (retained)

Variables: a_{ijt} (tank i holds grade j), I_{ijt} , in_{ijct} , out_{ijct} , r_{it} (receiving), z_{ict} (feeding).
 Constraints: one grade per tank; holdings need assignment $I_{ijt} \leq C_i a_{ijt}$; inventory balance; receiving flag; feeding link $out_{ijct} \leq M_i z_{ict}$ with data-driven $M_i = \min(\text{tank cap}, \text{max rate})$; **settling** $z_{ict}' + r_{it} \leq 1 \ \forall t' \in [t, t+\tau_i]$; feed-tank limit; CDU charge bounds; pipeline rate and vessel discharge; linear quality at CDU; symmetry breaking for truly identical tanks.
 Strengthening: flow-cover for receipts, settling-time clique cuts, minimum-run-length inequalities (added as static rows or separator plugins). These are used as **test families for our own cut framework** and to measure root-gap closure.

8.3 Mixed-grade tanks (bilinear; four-tier strategy retained)

Tier	Approach	Release
A (default)	Recipe blending, single-grade tanks $\rightarrow$ pure MILP	v1.0
B	Bilinear $P = q \cdot I$, $O = q \cdot f$; own spatial B&B with McCormick, bounded horizon	v2.0 (prototype v1.5)
C	Distributive recursion / successive LP; labelled local, no bound	v1.0 heuristic
D	Decomposition + piecewise-linear relaxation for long horizons	v2.0

Every scheduling plan is checked by forward simulation (FR-VER2) regardless of tier.

9. Numerical, verification and determinism specification

9.1 Tolerances (explicit; all overridable and logged)

Quantity	Value
LP primal / dual feasibility	1e-7
MIP feasibility	1e-6
Integrality	1e-6
Verifier row violation	$\leq 1e-6 \cdot \max(1, \ \text{row}\)$
Objective recomputation (compensated summation)	relative 1e-9
Cross-solver objective agreement	relative 1e-6
Coefficient range warning	ratio > 1e6
IPM complementarity / duality gap	1e-8 relative
Simplex pivot tolerance	1e-9 with threshold-relative test

9.2 Verifier (LP, QP and MILP differ)

- **MILP:** (1) primal feasibility vs the *original* model (post-postsolve); (2) integrality; (3) round integers, fix, re-solve LP to confirm feasibility; (4) recompute objective from raw data; (5) bound check: recompute $g(y, \lambda)$ from the root/tree certificates, confirm $\leq$ reported UB; (6) forward simulation for scheduling models.
- **LP / fixed-integer LP:** KKT (primal, dual feasibility; complementarity $\leq 1e-6$). Certifies optimality only for that integer assignment.
- **QP:** KKT with Q-terms + PSD check.

- Failure of any check → result withheld; status `NUMERICAL`.

9.3 Status codes and run tiers

Codes: `OPTIMAL`, `FEASIBLE`, `INFEASIBLE`, `UNBOUNDED`, `INF_OR_UNBD`, `NUMERICAL`, `TIME_LIMIT`, `WORK_LIMIT`, `INTERRUPTED`, each mapped to a CPLEX-style equivalent.

Tier	Purpose	Rules
T0 Opportunistic	Fastest; portfolio racing; not for publishing	Wall-clock limits allowed
T1 Deterministic (default for published plans)	Reproducible on same platform	Pinned container; fixed seeds; deterministic parallel B&B (fixed partition + ordered sync); fixed-order reductions on GPU, no atomics in determinism-critical kernels, explicitly stored A^T ; limits by iterations/nodes/work, never wall-clock ; canonical tie-break by lexicographic secondary objective on the fixed-integer LP
T2 Certified replay	Audit	Archive integer assignment, final basis, seeds, versions; CPU deterministic re-solve reproduces plan and explanations; cross-platform = verified equivalence within tolerance (feasibility 1e-6, objective 1e-9 rel.), not bit-exact

Acceptance: 100 T1 runs on one platform give identical plan hashes; T1 slowdown vs T0 is measured and reported. Cross-hardware bit-exactness is **not** promised.

10. Sensitivity and explanation (optional layer, retained in reduced form)

- **Attribution** (scope `local_at_incumbent`): duals in row units; reduced costs; breakeven $\kappa_{j,be} = \kappa_j + d_j$ for a non-slate variable at lower bound (quantity found by re-solve); cost/bound/RHS ranging from the basis.
- **Coefficient-embedded caps** (e.g., sulfur with RHS 0): $dz^*/dS = y \cdot \Sigma_{p|x_j} c$ while the basis is unchanged (envelope theorem); larger moves need parametric re-solves.
- **Contrastive "Why not X?"** (X-MILP style): encode alternative as constraints; add objective cutoff $c^T x \geq z^*$; if infeasible compute an IIS ("one of possibly several reasons"); if feasible-but-worse show plan diff and objective delta. LP-relaxation level only in v1.0; MILP-level IIS [v1.5]. Behaviour at refinery scale is **[VALIDATE]**.
- **Counterfactuals**: exact breakevens (LP) and bisection re-solves for MILP parameters, restricted to a domain-feasibility whitelist.
- **Trust display**: verifier checklist (pass/fail per check); no composite score; failed check → plan withheld.
- **ExplanationBundle**: JSON contract with `run_id`, `model_hash`, `status`, `objective`, `ub`, `gap`, `verifier{...}`, `duals[{row,value,unit,scope}]`, `reduced_costs[...]`, `ranging[...]`, `why_not[...]`.

11. Verification, validation and benchmarking plan

Reference oracles (external, harness-only, never linked): at least two established solvers (e.g., HiGHS and SCIP; a commercial solver where MRPL/academia holds a licence) for correctness cross-checks and performance comparison. Same hardware, same time/work budgets, report time **and** gap/feasibility, include cases where we lose.

11.1 Benchmark gates

Gate	Suite	Metric	Target [TARGET, set after Phase-1 measurement]
B1	Netlib LP (full collection incl. degenerate/ill-conditioned)	Solved to 1e-6 agreement; shifted geometric mean time vs reference	100% solved or correctly flagged; within ~3–5× of best open-source reference

Gate	conditioned Suite (members)	Metric	reference Target [TARGET, set after Phase-1 measurement]
B2	Mittelman LP benchmarks (subset by size class)	Solved/timeouts; time ratio	Subset solved; ratio reported (GPU vs CPU reported honestly)
B3	MIPLIB 2017 "benchmark" and "easy" sets	Instances solved to optimality within budget; shifted geometric mean time; primal-dual gap	Solved-count $\geq \sim 50\%$ of best open-source reference; rest with valid gaps
B4	QPLIB (convex continuous & discrete subsets)	Solved; objective agreement $1e-6$	Convex continuous subset solved
B5	Refinery/industrial suite (§8)	Time-to-verified-gap; time-to-1%-of-best-known	Published vs reference oracles; escalation rule below
B6	Scale ladder	$10^3 \rightarrow 10^4 \rightarrow 10^5 \rightarrow 10^6$ rows/cols LP (sparse, generated + public)	Memory/time curve; IPM+crossover and PDHG demonstrated at $\geq 10^6$ nonzeros scale

Escalation rule: if median time-to-verified-gap on ≥ 3 Level-2 instances exceeds $10\times$ the best reference, switch effort to decomposition/LNS-first (rolling horizon, fix-and-relax) and GPU heuristics.

11.2 Robustness demonstration (PS-specific)

Challenge	Test set / generator	Pass criterion
Highly degenerate LPs	Assignment/transportation-type, Netlib degenerate members, generated degenerate vertex problems	No cycling; iteration count reported with/without perturbation
Ill-conditioned matrices	Netlib (e.g., badly scaled members), generated $\text{cond}(B)$ up to $1e12$	Correct status or verified solution; scaling/refinement logs
Weak LP relaxations	Big-M scheduling, fixed-charge network	Root-gap closure

Challenge	design Test set / generator	with/without cuts Pass criterion reported
Infeasible/unbounded	Netlib infeasible set, fuzz	Correct status + certificate (ray / Farkas / IIS)
Solver-killing cases	Fuzzed random models, kill-process tests	Platform never crashes; last verified incumbent preserved

11.3 Other V&V

Level	Method	Pass criterion
Equation	Each spec item on hand- computed toy cases (App. A)	Exact to 1e-9
Dual bound	Random LPs: $g(y, \lambda) \geq \text{true}$ optimum for any $y \geq 0$	0 violations
Sensitivity	Finite differences vs duals / sulfur sensitivity	Within tolerance
Determinism	100 repeated T1 runs; T2 replay; cross-platform check	Identical hash (T1/T2); tolerance equivalence across platforms
Cuts	Root-gap closure and node count with/without each family	Improvement reported per family
Domain	Replay ≥ 3 historical months in comparison mode [VALIDATE]	Within agreed tolerance
Regression/CI	Nightly benchmark	No >10% regression; no result drift
Security	Threat model, dependency scan, pen-test	No critical findings
Independent review	OR specialist review of §7–§9	Sign-off before pilot

12. Non-functional requirements

Area	Requirement
Scale	Thousands to millions of variables/constraints (sparse); memory-aware data structures; out-of-core not required v1.0
Performance	Targets set by Phase-1 measurement; GPU claims only where measured; interactive what-ifs < 200 ms on small models
Reliability	Process isolation; watchdog; append-only run archive
Portability	Linux (RHEL/Ubuntu), containers, CUDA 12.x for GPU; CPU-only mode always functional ; C++17/20 core
Security	Air-gapped install, RBAC/SSO for service mode, encryption at rest/in transit, tamper-evident audit log, SBOM, signed releases
Sovereignty	No external calls; no telemetry; any language model hosted locally (v1.5 optional)
Transparency	Full source, design notes per algorithm, provenance log, extension guide
Licensing	Own code under chosen permissive licence [VALIDATE]; dependency audit (§3)
Accessibility (if UI built)	Colour-blind-safe palette, screen-reader lists, 2D fallback; Hindi/English labels [C]
Governance	ISO/IEC 42001-aligned practices where applicable [VALIDATE]

13. Releases, delivery plan, metrics, risks

13.1 Releases

Release	Scope
SIH	Own MPS reader, presolve (basic), sparse LU, dual simplex (+perturbation),
prototype	basic IPM, B&B with heuristics, branching, Crossover/MIP/crossover, ...

prototype Release (MVP)	Scope basic IPM, B&B with pseudocost branching + Gomory/MIR/cut cuts + rounding/diving, verifier, CLI, Netlib subset + small MIPLIB subset benchmark report vs ≥ 1 reference solver, L1 crude model demo
v1.0	Full LP (primal/dual simplex, IPM + crossover), convex QP (IPM), MILP with full cut/heuristic set, T0/T1/T2, deterministic parallel, C/Python API, benchmark gates B1–B5, robustness suite, L2 single-grade scheduling, sensitivity/IIS, comparison mode
v1.5	GPU PDHG LP + GPU heuristics promoted if beneficial, convex MIQP, MILP-level IIS, LLM narration (optional), UI dashboard/3D trace, expression-graph layer
v2.0	Spatial B&B (bilinear/MINLP), NLP local solver, decomposition for long-horizon pooled tanks, stochastic/CVaR, multi-GPU

13.2 Phases

0 Discovery, clean-room setup, benchmark harness, G0 (4w) · 1 LP core: LA, presolve, dual simplex, IPM (10w) · 2 MILP core: B&B, cuts, heuristics, verifier (10w) · 3 QP, parallel/determinism tiers, robustness suite (8w) · 4 GPU module + router + benchmarks B1–B6 (8w) · 5 Application models, sensitivity/IIS, shadow pilot with MRPL (12w) · 6 Hardening, security, independent review (8w). Suggested team: 3 optimization/OR (LP, MIP, QP), 2 C++/CUDA, 1 numerical linear algebra, 1 test/benchmark/QA-DevOps, 1 refinery-planning SME, 1 API/frontend, 1 product owner.

13.3 Success metrics

- 100% of published results pass the verifier.
- Netlib fully handled (solved or correct certificate); MIPLIB/QPLIB solved-counts per B3/B4 targets.
- Published comparison vs ≥ 2 established solvers, including losses.
- 100-run T1 identical-hash test passes.
- Refinery deviation vs incumbent tool within agreed tolerance [VALIDATE].
- Zero solver-library dependencies (G0) at every release.

13.4 Risks

Risk	Mitigation
------	------------

Scope: from-scratch CPLEX-class engine is a multi-year effort

Mitigation targets; MVP first; prioritize dual simplex + B&B + core cuts; honest benchmark reporting

Simplex numerical instability (degeneracy, ill-conditioning)

Perturbation, Harris ratio test, stability-triggered refactor, scaling, IPM fallback, fuzzing

Slow open-source-class MIP ($\approx 10\times$ gap to commercial on hard instances)

Strong formulations, cuts library, decomposition/LNS, GPU heuristics, portfolio, anytime KPI, oracle-ratio escalation

GPU LP not faster than best CPU LP

Router enables GPU only where measured; GPU restricted to Candidate tier

Ambiguity over "from scratch" (BLAS/cuSPARSE)

Own kernels on critical path; vendor primitives swappable; clarify in Phase 0

Determinism broken by driver/library/hardware changes

Pinned containers, environment fingerprint, T2 replay, tolerance equivalence

Beta/experimental defect reaching users

Off published path, isolation, verifier gate, promotion criteria

Bilinear models too slow

Tier A default; Tier B bounded horizon; Tier C flagged local; v2.0 decomposition

Planner distrust

Comparison mode, shadow pilot, verifier checklist

Data access to incumbent tool

CSV/Excel adapter; open literature data fallback

Search trace mistaken for causality

Label + link to explanation panel

IP contamination claim

Clean-room policy, provenance log, design notes

13.5 Known limitations (open)

1. L1 yields are linear aggregates; real yields depend on cut points/unit modes (Level 3).
2. Emission constraints are feed-based proxies until energy balances exist.
3. Duals and breakevens are local to the fixed integers.
4. Mixed-grade (bilinear) tank optimization arrives with spatial B&B (v2.0).
5. Proof certificates are optional/post-v1.0 and exclude presolve initially.
6. All performance targets are unmeasured; CPU MIP B benefit unproven

6. All performance targets are unmeasured, GPU MILP benefit unproven.
7. Independent expert review of §7–§9 outstanding.
8. Open-source-vs-commercial gap estimates come from discussion-level sources (order of magnitude), not a controlled study.

13.6 Open questions for MRPL / SIH mentors

Strictness of "from scratch" (vendor BLAS/cuSPARSE allowed?); which reference solvers are acceptable for comparison; expected instance sizes and time budgets; current planning tool, model size and turnaround; planning windows; CDU configuration and crude–CDU allowability; tank grade practice; emission cap definitions; approved hardware/OS/air-gap policy; GPU availability; acceptance tolerance; validation sign-off owner.

14. References (status)

- HiGHS docs (options, ranging, PDLP has no basis/crossover) — reference oracle and comparison only.
- SCIP Optimization Suite 10.0 report, arXiv 2511.18580; SCIP 8 MINLP paper, arXiv 2301.00587 — reference for benchmark comparison and algorithm background.
- Lu, Yang, cuPDLP.jl; Lu, Peng, Yang, cuPDLPx, arXiv 2507.14051; Lu et al., cuPDLP-C, arXiv 2312.14832; Applegate et al., PDLP (2021); Chambolle–Pock (2011) — mathematical background for own PDHG implementation.
- Çördük, Sielski, Boucher, Aatish, GPU-Accelerated Primal Heuristics for MIP, arXiv 2510.20499 — background for GPU heuristics.
- Fix-and-propagate with low-precision first-order LP solutions, Math. Programming Computation 2026 — exists; details [VERIFY].
- Lera-Leri et al., X-MILP, arXiv 2507.13007 / Springer LNAI 15936; Lefebvre & Schmidt, EJOR 2026; Kurtz, Birbil, den Hertog, EJOR 2025; Engelhardt et al., arXiv 2510.17624 — explanation methods.
- Standard textbooks/papers to be cited per module design note (simplex/dual steepest edge, Forrest–Tomlin, Mehrotra, MIR/Gomory cuts, reliability branching, feasibility pump/jump) [add exact citations in Phase 0].
- IBM CPLEX determinism documentation; NVIDIA cuSPARSE reproducibility documentation — determinism standards.
- Benchmarks: MIPLIB 2017, Netlib LP, Mittelmann benchmarks, QPLIB.
- MRPL facts: mrpl.co.in; Business Standard 26 Jul 2023; IIFL; India Business Journal.

- Removed as unverifiable: "Batched FOM ICML 2026 (12–500×)"; "5,000× cuOpt speedup".

Appendix A — Hand-checked equation tests (retained)

A1 Dual bound (max). $\max x_1+x_2$ s.t. $x_1+2x_2 \leq 4$, $0 \leq x \leq 3$. Optimum $x=(3, 0.5)$, value 3.5.
 $y=0.5$: $r=(0.5, 0)$, $g=4 \cdot 0.5+0.5 \cdot 3=3.5$ (tight). $y=0$: $r=(1, 1)$, $g=6 \geq 3.5$. $y=1$: $r=(0, -1)$, $g=4 \geq 3.5$. A wrong min-form box term at $y=0$ gives $0 < 3.5$ (invalid) — the test that catches that defect.

A2 Sulfur with density. A: $s=0.3$, $p=0.12$, $x=60 \rightarrow$ mass 7.2; B: $s=0.9$, $p=0.14$, $x=40 \rightarrow$ mass 5.6. Mass-avg $S = (2.16+5.04)/12.8 = 0.5625$ wt%. With $S_{\max}=0.5$: $(0.3-0.5)(7.2)+(0.9-0.5)(5.6) = +0.80 > 0 \rightarrow$ violated.
 Cross-check $12.8 \cdot 0.0625 = 0.80 \checkmark$.

A3 Gravity vs API. $API \geq 31 \rightarrow SG_{\max} = 141.5/162.5 = 0.870769$. Crudes API 33 (SG 0.860182) and 29 (SG 0.881620), 50/50 by volume: blend SG = 0.870901 $\rightarrow API \approx 30.975 < 31$. Constraint value $+0.0132 > 0 \rightarrow$ violated $\checkmark$. Linear API averaging (31.0) would wrongly pass.

A4 Breakeven. $\max 3x_1+2x_2+x_3$ s.t. $x_1+x_2+x_3 \leq 4$, $0 \leq x \leq 3$. Optimum (3,1,0), value 11, $y=2$.
 Reduced costs $d_1=1$ (at upper), $d_2=0$ (basic), $d_3=-1$. Breakeven coefficient for $x_3 = 2$: tie at 2.0; enters at 2.01 $\checkmark$.

A5 Sulfur sensitivity. Perturb $S_{\max}$ by $+0.01$; finite-difference Δz equals $y \cdot \Sigma \rho x \cdot 0.01$ while basis is unchanged.

A6 Settling. One tank, $\tau=2$, receipt at $t=3$: $z_3, z_4, z_5=0$ enforced; feeding at $t=6$ allowed. Fails for the vacuous "sum $\leq \tau+1$ " form.

A7 Symmetry breaking. Two identical tanks: optimal objective equal with/without the cut; with the cut tank-1 receipts $\geq$ tank-2.

A8 Degenerate LP (new). $\max x_1+x_2$ s.t. $x_1 \leq 1$, $x_2 \leq 1$, $x_1+x_2 \leq 2$ (three constraints active at (1,1)). Dual simplex with perturbation must terminate, value 2, and return a valid dual (e.g., $y=(1,1,0)$ or $(0,0,1)$); verifier KKT passes for whichever is returned.

A9 Convex QP (new). $\min \frac{1}{2}(x_1^2 + x_2^2) - x_1 - 2x_2$ s.t. $x_1 + x_2 \leq 1, x \geq 0$. KKT solution $x = (0, 1)$, objective -1.5 ; IPM result must match to $1e-8$, with multiplier $\lambda=1$ on the coupling row.

A10 Cut validity (new). For $5x_1 + 6x_2 \leq 8, x \in \{0, 1\}^2$, the set $\{1, 2\}$ is a cover ($5+6 = 11 > 8$), so $x_1 + x_2 \leq 1$ is valid: enumeration shows $(0,0), (1,0), (0,1)$ are feasible and $(1,1)$ is not. The LP point $(1, 0.6)$ has $5+3.6 = 8.6 > 8$, so take $(1, 0.5)$: $5+3 = 8$ is LP-feasible with $x_1 + x_2 = 1.5 > 1$, so the cut removes it. Every cut family gets an exhaustive-enumeration validity test on toy cases like this.

